

tiezerk Repository Development Plan
Pinnacle BBS Backbone Architecture
changed ashkharh to tiezerk..navigo
tiezerk fits universal concepts better ..

****Organization****: eaprimel/ashkharh
****Tier****: 1 (Universal Hub)
****Purpose****: Primary interaction space, backbone nodes, BBS system

Repository Structure

Primary Repos (Clone into Organization)

...

ashkharh/
├── CUBICULUM/ # Primary interaction space (BBS core)
├── gubernaculum/ # Governance (RETINUE advisors)
├── runexusiam/ # Play/imaginary (development/testing)
├── consortium/ # Create/collaborative
├── unexusiam/ # Work/productivity/missions
├── nexusiam/ # Synergy (play + development mix)
├── void-marrow/ # Connection phenomena (wormholes, void tubes)
└── dialup/ # Dialup protocols, special access

...

CUBICULUM (Primary BBS)

****Role****: Central interaction space

****Content****:

- BBS system core
- Universal concepts
- Navigation hub
- User authentication
- Message boards
- Real-time interaction protocols

****Key Files****:

- `README.md` - Welcome, overview, getting started
- `NAVIGATION.md` - How to traverse ashkharh network
- `DIALUP.md` - Connection protocols
- `WITNESSMARK.md` - Transaction protocol documentation
- `bbs-core/` - Main BBS engine code
- `routes/` - Connection paths to other repos

gubernaculum (Governance)

****Role****: RETINUE advisors, governance layer

****Content****:

- Advisory protocols
- Decision frameworks
- Ethical guidelines
- Chain of custody rules
- Review processes

****Key Files****:

- `README.md` - Governance overview
- `RETINUE.md` - Advisor structure
- `ETHICS.md` - Ethical frameworks
- `REVIEW\_PROTOCOL.md` - How content is reviewed
- `advisors/` - Individual advisor profiles

runexusiam (Play/Imaginary)

****Role****: Play, mental realm, development/testing

****Content****:

- Game mechanics
- Testing environments
- Experimental features
- Play missions
- Development sandboxes

****Key Files****:

- `README.md` - Play realm overview
- `MISSIONS.md` - Available play missions
- `TESTING.md` - Test environments
- `games/` - Game implementations
- `experiments/` - Feature experiments

consortium (Create/Collaborative)

****Role****: Creative development, collaboration

****Content****:

- Collaborative tools
- Creative frameworks
- Project templates
- Shared resources

****Key Files****:

- `README.md` - Collaboration overview
- `PROJECTS.md` - Active projects
- `TOOLS.md` - Available tools
- `templates/` - Project templates

- `resources/` - Shared assets

unexusiam (Work/Missions)

****Role****: Productivity, work tasks, mission coordination

****Content****:

- Mission structures
- Task management
- Productivity tools
- Workflow automation

****Key Files****:

- `README.md` - Work realm overview
- `MISSIONS.md` - Active missions
- `WORKFLOWS.md` - Automation patterns
- `tasks/` - Task definitions
- `tools/` - Productivity utilities

nexusiam (Synergy)

****Role****: Mixture of play + development, finding missions

****Content****:

- Hybrid content (play + work)
- Mission discovery
- Synergy protocols
- Cross-realm bridges

****Key Files****:

- `README.md` - Synergy overview
- `PLAY\_MISSIONS.md` - Play-oriented missions
- `DEV\_MISSIONS.md` - Development missions
- `BRIDGES.md` - Cross-realm connections
- `synergy/` - Hybrid content

void-marrow (Connection Phenomena)

****Role****: Wormholes, void tubes, cross-system links

****Content****:

- Connection protocols
- Wormhole navigation
- Void tube mechanics
- Cross-BBS bridges

****Key Files****:

- `README.md` - Void marrow overview

- `WORMHOLES.md` - Wormhole protocols
- `VOID\_TUBES.md` - Void tube mechanics
- `CONNECTIONS.md` - External BBS links
- `protocols/` - Connection implementations

dialup (Dialup Protocols)

****Role****: BBS dialup, special access, random connections

****Content****:

- Dialup sequences
- Connection handshakes
- Special access protocols
- Random connection mechanics
- Visionary access paths

****Key Files****:

- `README.md` - Dialup overview
- `HANDSHAKE.md` - Connection protocols
- `SPECIAL\_ACCESS.md` - Visionary paths
- `RANDOM.md` - Random connection mechanics
- `sequences/` - Dialup implementations

Branch Protection Strategy

Public-Facing Branches

****Branch Name****: `public` or `release`

****Purpose****: Public-facing content, stable releases

****Protection Rules****:

```yaml

Branch Protection Rules for 'public':

- Require pull request reviews before merging: YES
- Required approving reviews: 2
- Dismiss stale reviews when new commits are pushed: YES
- Require review from Code Owners: YES
- Require status checks to pass before merging: YES
- Require branches to be up to date before merging: YES
- Status checks that must pass:
  - \* build-test
  - \* lint-check
  - \* link-validator (for .md files)
- Require conversation resolution before merging: YES

- Require signed commits: YES (if possible)
- Include administrators: NO (admins can bypass for emergencies)
- Restrict pushes: YES
- Only allow specified users/teams to push:
  - \* @eaprime1 (you)
  - \* @ashkharh-admins (team)
- Allow force pushes: NO
- Allow deletions: NO

...

---

#### ### Main/Default Branch

**\*\*Branch Name\*\*:** `main` or `master`

**\*\*Purpose\*\*:** Development branch, active work

**\*\*Protection Rules\*\*:**

```yaml

Branch Protection Rules for 'main':

- Require pull request reviews before merging: YES
- Required approving reviews: 1
- Dismiss stale reviews when new commits are pushed: NO
- Require status checks to pass before merging: YES
- Require branches to be up to date before merging: NO
- Status checks that must pass:
 - * basic-build
- Require conversation resolution before merging: NO
- Require signed commits: NO (more flexible for rapid dev)
- Include administrators: NO
- Restrict pushes: YES
- Allow specified users/teams:
 - * @eaprime1
 - * @ashkharh-developers (team)
 - * @consortium-members (team)
- Allow force pushes: YES (for specific people only)
- Specify who can force push:
 - * @eaprime1

- Allow deletions: NO
...

Markdown as Pages

Using .MD Files as Web Pages

YES - Markdown files can serve as pages through:

Option 1: GitHub Pages

- Enable GitHub Pages in repo settings
- Set source to `main` or `public` branch
- Choose folder: `/ (root)` or `/docs`
- Markdown files auto-render as HTML
- Access via: `https://eaprimel.github.io/ashkharh/`

Option 2: Jekyll (GitHub's static site generator)

```yaml

# \_config.yml in repo root  
theme: jekyll-theme-minimal  
title: ASHKHARH  
description: Pinnacle BBS System  
...

#### \*\*Option 3: Custom Static Site Generator\*\*

- Use MkDocs, Hugo, or Docusaurus
- Convert .md → HTML via GitHub Actions
- Deploy to GitHub Pages or Vercel

---

## ## Workflows for Document Launch Readiness

### ### GitHub Actions Workflows

#### \*\*Workflow 1: Launch Readiness Check\*\*

```yaml

.github/workflows/launch-readiness.yml
name: Launch Readiness Check

on:

pull_request:

branches: [public]

jobs:

readiness:

```
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

- name: Check WitnessMark
run: |
# Verify document has witnessmark metadata
python scripts/check_witnessmark.py

- name: Validate Links
run: |
# Ensure all internal links work
npm run link-check

- name: Content Quality
run: |
# Check spelling, grammar, formatting
npm run quality-check

- name: Prime Readiness Score
run: |
# Calculate readiness score
python scripts/prime_score.py
````
```

**\*\*Workflow 2: Refinement Pipeline\*\***

```
``yaml
.github/workflows/refinement.yml
name: Refinement Pipeline
```

```
on:
push:
branches: [main]
```

```
jobs:
refine:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3
```

```
- name: Run Tests
run: |
Test system for content
npm test
```

```
- name: Generate Metadata
run: |
Auto-generate frontmatter/metadata
```

python scripts/generate\_metadata.py

```
- name: Update Progress
run: |
Update Linear with progress
node scripts/update_linear.js
...
```

---

## Reference Document: Tools, Plugins, Tasks, Agents

**\*\*File\*\*:** `TOOLKIT\_REFERENCE.md` (in CUBICULUM)

``markdown

# ASHKHARH Toolkit Reference  
Quick reference for all tools, plugins, tasks, and agents

## Development Tools

### Core Languages

- **\*\*Python 3.11+\*\*** - Backend, automation scripts
- **\*\*JavaScript/Node.js\*\*** - BBS frontend, real-time features
- **\*\*TypeScript\*\*** - Type-safe components
- **\*\*Bash\*\*** - System automation

### Frameworks

- **\*\*React\*\*** - UI components (if web interface)
- **\*\*Flask/FastAPI\*\*** - Backend APIs
- **\*\*Socket.io\*\*** - Real-time BBS communication

### Testing

- **\*\*pytest\*\*** - Python testing
- **\*\*jest\*\*** - JavaScript testing
- **\*\*cypress\*\*** - End-to-end testing

## GitHub Actions Plugins

### Essential

- `actions/checkout@v3` - Checkout code
- `actions/setup-python@v4` - Python environment
- `actions/setup-node@v3` - Node environment
- `peaceiris/actions-gh-pages@v3` - Deploy to GitHub Pages

### Quality

- `github/super-linter@v4` - Multi-language linting
- `reviewdog/action-markdownlint@v0` - Markdown linting
- `gaurav-nelson/github-action-markdown-link-check@v1` - Link validation



#### ### Automation

- `stefanzweifel/git-auto-commit-action@v4` - Auto-commit changes
- `peter-evans/create-pull-request@v5` - Auto PR creation

### ## Key Tasks

#### ### Development

- `npm run dev` - Start development server
- `npm run build` - Build for production
- `npm test` - Run test suite
- `npm run lint` - Lint all code

#### ### Documentation

- `npm run docs:build` - Generate docs
- `npm run docs:serve` - Preview docs locally

#### ### Deployment

- `npm run deploy:staging` - Deploy to staging
- `npm run deploy:production` - Deploy to public branch

### ## AI Agents/Tools

#### ### Claude (Navigo Nexusuxen 🦋)

- Strategic planning
- Content refinement
- Architecture design
- Documentation generation

#### ### Gemini (Navigo Gemini 🟡)

- Data analysis
- Pattern recognition
- Multi-perspective synthesis

#### ### Linear Integration

- Issue tracking
- Project management
- Shadow development coordination

#### ### GitHub Copilot

- Code completion
- Pattern suggestions
- Automation assistance

### ## Linear Tasks

#### ### Standard Labels

- `shadow` - Shadow development phase

- `orbit` - Orbiting, nearly ready
- `ready` - Ready for launch
- `launched` - Live in PRIMAL
- `witnessed` - Has WitnessMark

#### ### Project Templates

- Choose Your Own Adventure
- BBS Feature Development
- Documentation Sprint
- Refinement Cycle

#### ## Common Commands

##### ### Git Operations

```
```bash
```

```
# Create feature branch
```

```
git checkout -b feature/new-mission
```

```
# Commit with witnessmark
```

```
git commit -m "feat: new mission [WITNESSED]"
```

```
# Push and create PR
```

```
git push -u origin feature/new-mission
```

```
gh pr create --base main
```

```
```
```

##### ### Testing

```
```bash
```

```
# Run full test suite
```

```
npm test
```

```
# Run specific test
```

```
npm test -- --grep "dialup"
```

```
# Run with coverage
```

```
npm run test:coverage
```

```
```
```

##### ### Documentation

```
```bash
```

```
# Generate API docs
```

```
npm run docs:api
```

```
# Update README
```

```
npm run docs:readme
```

```
# Check links
```

```
npm run docs:links
```

...

Emergency Procedures

Rollback Release

```
```bash
git revert
git push origin public
```
```

Fix Broken Links

```
```bash
npm run links:fix
git commit -am "fix: repair broken links"
```
```

Force Rebuild

```
```bash
rm -rf node_modules
npm install
npm run build
```
...
```

Next Steps

1. **Create repos** for each component
2. **Set up branch protection** as specified
3. **Enable GitHub Pages** for public-facing content
4. **Configure GitHub Actions** for launch readiness
5. **Create TOOLKIT_REFERENCE.md** in CUBICULUM
6. **Link repos** via void-marrow connections

Status: Architecture documented

Ready for: Repository creation, branch protection setup